DUOC UC

Desarrollo Cloud Native I

Despliegue Cloud Native de ClassFlow en AWS

Informe de Evaluación EVA2

Backend de microservicios Spring Boot y frontend React desplegados en Amazon EC2, con HTTPS, Microsoft Entra ID y RabbitMQ

Alumno: Evens Reneus

Asignatura: Desarrollo Cloud Native I

Evaluación: EVA2

Fecha: 5 de noviembre de 2026

Índice

Resumen **3**

1. Introducción y objetivos **3**

2. Sistema ClassFlow y sus microservicios **3**

2.1 Repositorios 4

2.2 Vista lógica 5

2.3 Patrones de arquitectura aplicados 5

2.4 Estructura interna del código 6

2.5 Despliegue alternativo en Kubernetes 6

3. Arquitectura de despliegue en AWS **7**

4. Tecnologías utilizadas **8**

5. Despliegue del backend **8**

6. Despliegue del frontend **9**

7. Autenticación con Microsoft Entra ID **10**

8. Mensajería asíncrona con RabbitMQ **10**

9. Problemas encontrados y soluciones **12**

10. Pruebas y evidencias **13**

11. Seguridad, operación y mantenimiento **13**

12. Conclusiones y trabajo futuro **14**

Resumen

ClassFlow quedó desplegado y operativo en AWS: backend y frontend en dos instancias EC2, con HTTPS, inicio de sesión con Microsoft Entra ID y mensajería asíncrona con RabbitMQ.

Campo	Detalle
Asignatura	Desarrollo Cloud Native I
Evaluación	EVA2
Alumno	Evens Reneus
Institución	Duoc UC
Repositorio backend	github.com/Ereneus/ClassFlowBack
Repositorio frontend	github.com/Ereneus/ClassFlowFront
URL frontend	https://100-60-245-109.sslip.io
URL API	https://32-201-134-233.sslip.io/api

1. Introducción y objetivos

Este informe documenta el despliegue en la nube de ClassFlow, un sistema de gestión escolar construido con microservicios. El trabajo se realizó sobre AWS Academy Learner Lab, región us-east-1.

Objetivo general: desplegar la aplicación completa en Amazon EC2 de forma segura, reproducible y accesible desde internet.

Objetivos específicos:

1. Contenerizar y orquestar los 7 servicios del backend y sus bases de datos con Docker Compose.
2. Desplegar el frontend en una instancia separada del backend.
3. Exponer ambos servicios con HTTPS y certificados válidos de Let's Encrypt.
4. Integrar la autenticación con Microsoft Entra ID (OAuth 2.0 / OpenID Connect).
5. Habilitar la mensajería asíncrona entre microservicios con RabbitMQ, con confirmación manual (ACK) y Dead Letter Queue.
6. Validar el funcionamiento de punta a punta con pruebas reales.

2. Sistema ClassFlow y sus microservicios

ClassFlow separa cada dominio escolar en un microservicio Spring Boot con su propia base de datos PostgreSQL (patrón *database per service*). Toda petición externa entra por el API Gateway. El frontend es una SPA React/Vite que vive en un repositorio aparte.

Servicio	Puerto interno	Responsabilidad	Base de datos

Servicio	Puerto interno	Responsabilidad	Base de datos
api-gateway	8080	Punto de entrada único (Spring Cloud Gateway), CORS y validación de tokens Entra	—
bff	8086	Backend-for-Frontend: agrega datos para el dashboard	—
ms-auth	8081	Usuarios, roles y vínculo con identidades de Entra	auth-db
ms-academic	8082	Cursos, asignaturas, evaluaciones y notas	academic-db
ms-assistance	8083	Asistencia y anotaciones	assistance-db
ms-message	8084	Mensajes y anuncios; publica eventos en RabbitMQ	message-db
ms-notification	8085	Notificaciones y correo (Brevo SMTP); consume eventos de RabbitMQ	notification-db
glitchtip	8000	Seguimiento de errores compatible con Sentry	glitchtip-db + Valkey
frontend	80	SPA React/Vite servida por nginx	—

Las migraciones de esquema se gestionan con Flyway y la documentación de cada API se publica con springdoc (Swagger).

2.1 Repositorios

Repositorio	Último commit	Contenido
Ereneus/ClassFlowBack	`8701bbd`	Proyecto Maven multimódulo (7 módulos), un `Dockerfile` por servicio, `docker-compose.yml`, manifiestos `k8s/` y guía `docs/ENTRA-AWS-HANDOFF.md`
Ereneus/ClassFlowFront	`8a54adb`	SPA React + Vite + TypeScript, `Dockerfile` multi-stage (Node 22 → nginx) y `nginx.conf`

2.2 Vista lógica

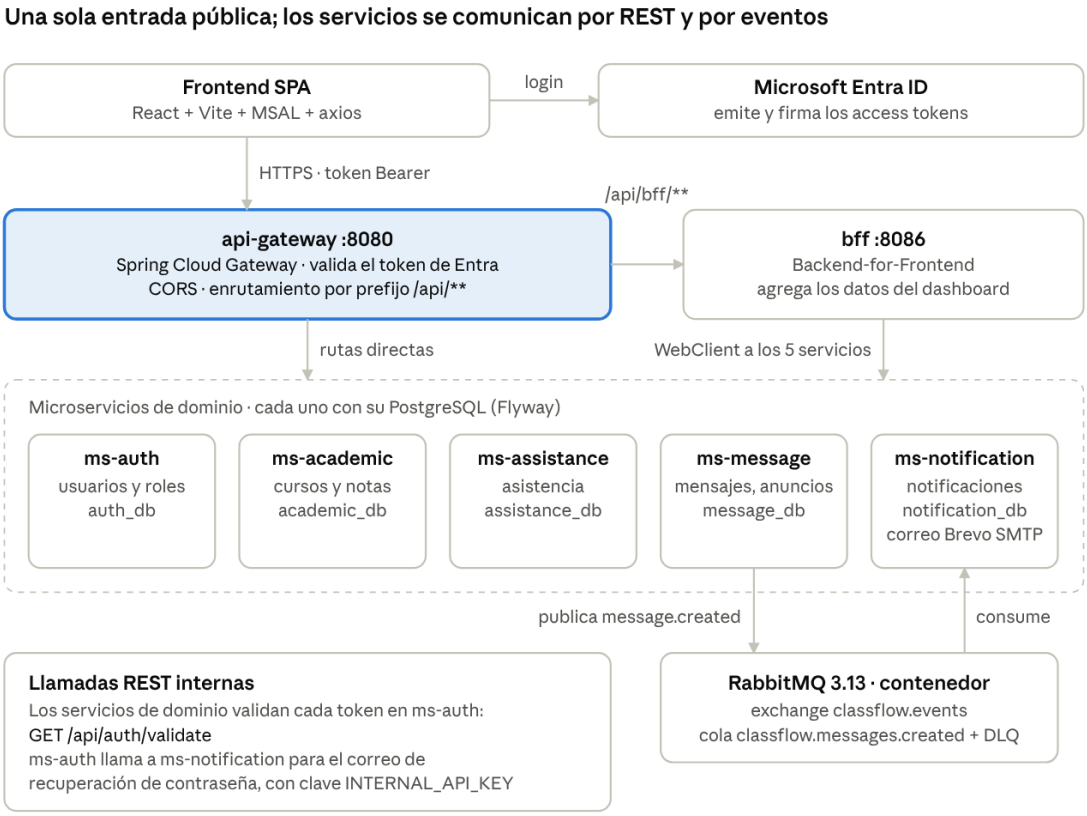


Figura 1. Vista lógica de ClassFlow · 1 gateway, 1 BFF, 5 microservicios

El navegador solo habla con el gateway. Las consultas simples van directo a cada microservicio; el dashboard pasa por el BFF, que reúne datos de los 5 servicios en una sola respuesta.

2.3 Patrones de arquitectura aplicados

Patrón	Dónde se aplica	Beneficio
API Gateway	`api-gateway` enruta por prefijo: `/api/auth`, `/api/courses`, `/api/attendance`, `/api/messages`, `/api/notifications`, `/api/bff`	Una sola entrada pública, CORS y seguridad centralizados
Backend-for-Frontend	`bff` consulta en paralelo a los 5 servicios con WebClient reactivo	El dashboard se arma con una sola llamada del navegador
Database per service	5 PostgreSQL independientes, esquema versionado con Flyway	Cada servicio evoluciona y escala sin acoplarse a otro
Arquitectura orientada a	`ms-message` publica `message.created`; `ms-`	Desacople temporal: si el consumidor cae, los eventos

Patrón	Dónde se aplica	Beneficio
eventos	`notification` consume, con 3 reintentos y DLQ	esperan en la cola
Federación de identidad	Gateway, BFF y ms-auth como *OAuth2 Resource Server* de Microsoft Entra ID	Sin contraseñas propias en producción; tokens firmados y verificables
Validación delegada	ms-academic, ms-assistance, ms-message y ms-notification llaman a `GET /api/auth/validate`	Un único lugar resuelve el usuario interno y su rol
Configuración externalizada	Perfiles Spring (`docker`, `entra`) y variables en `.env`	La misma imagen corre en local, Docker y Kubernetes
Contenedores inmutables	Dockerfile multi-stage (Maven → JRE alpine, usuario sin privilegios)	Imágenes pequeñas y reproducibles
Health checks	`pg_isready` en las bases y Spring Actuator en los servicios	Arranque ordenado y diagnóstico rápido

2.4 Estructura interna del código

Cada microservicio sigue la misma arquitectura en capas, en el paquete `com.ohiggins.classflow.<servicio>`:

1. `controller`: endpoints REST documentados con Swagger.
2. `service`: reglas de negocio.
3. `repository`: acceso a datos con Spring Data JPA.
4. `entity` y `dto`: modelo persistente y objetos de intercambio.
5. `security`, `config` y `exception`: filtro de autenticación, configuración y manejo global de errores.

El frontend se organiza en `pages` (vistas por rol: admin, teacher, portal de estudiante y apoderado), `services` (clientes HTTP con axios), `context` y `router`, y `config` (MSAL y URL de la API). Los roles del sistema son `ADMINISTRATOR`, `TEACHER`, `GUARDIAN` y `STUDENT`.

2.5 Despliegue alternativo en Kubernetes

El repositorio incluye manifiestos en `k8s/` para un clúster: namespace `classflow`, ConfigMap, Secret, un Deployment y Service por microservicio, las 5 bases de datos y el frontend. El Ingress de Traefik v3 usa un middleware *ForwardAuth* que valida cada petición contra `ms-auth /api/auth/validate` antes de llegar al gateway. Para esta entrega se eligió Docker Compose en EC2 por los límites de AWS Academy.

3. Arquitectura de despliegue en AWS

La solución usa dos instancias EC2 en us-east-1, cada una con IP elástica y su propio Caddy como única puerta HTTPS. El navegador descarga el sitio desde la EC2 del frontend y llama a la API directamente en la EC2 del backend con el token de Entra.

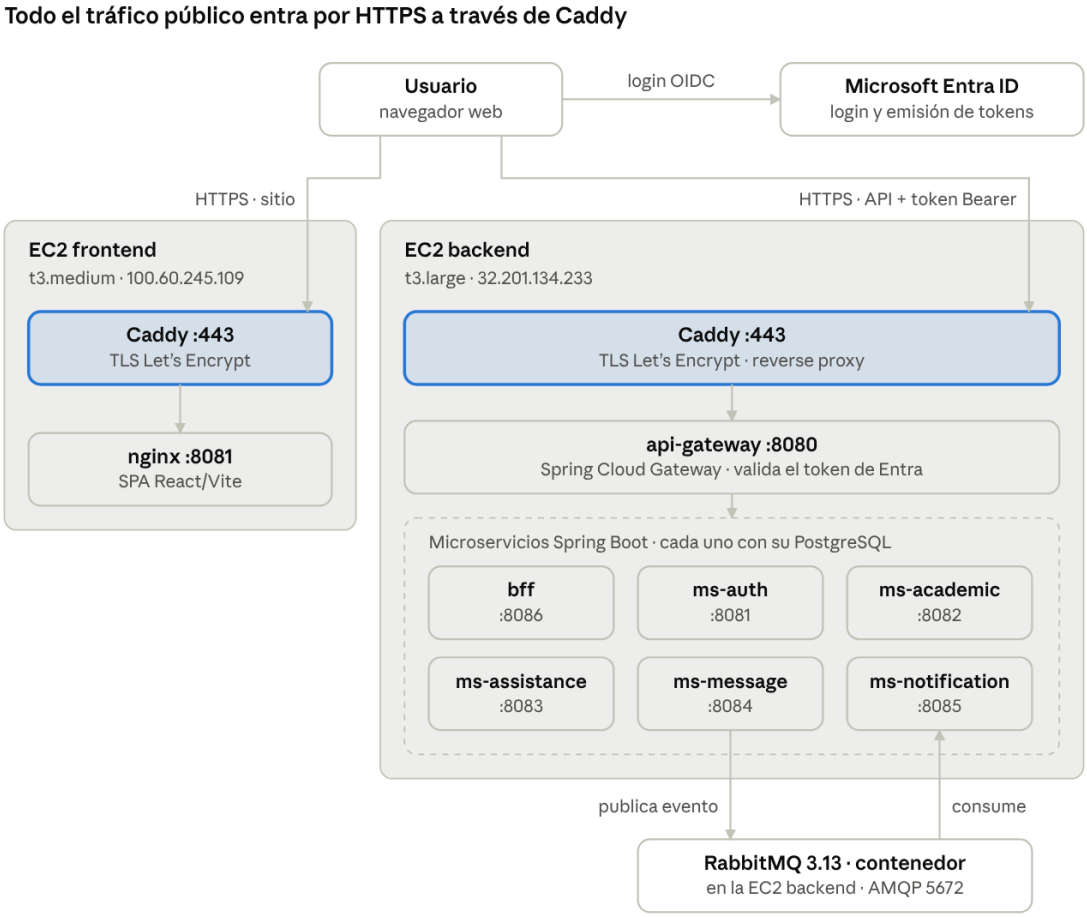


Figura 2. Arquitectura de despliegue en AWS · 2 EC2, 1 broker externo

Los nombres `*.sslip.io` resuelven a la IP incrustada en el nombre, lo que permite obtener certificados de Let's Encrypt sin comprar un dominio.

Instancia	Tipo	IP elástica	URL pública	Reglas de entrada
classflow-backend	t3.large, 30 GiB gp3	32.201.134.233	https://32-201-134-233.sslip.io	22, 80, 443 (0.0.0.0/0); 8000 pendiente, solo para la IP del administrador
classflow-frontend	t3.medium, 20 GiB gp3	100.60.245.109	https://100-60-245-109.sslip.io	22, 80, 443 (0.0.0.0/0)

El puerto 80 queda abierto porque Let's Encrypt valida el dominio por HTTP (desafío HTTP-01); Caddy redirige ese tráfico a HTTPS.

4. Tecnologías utilizadas

Capa	Tecnología	Uso en el proyecto
Lenguaje y framework	Java 25, Spring Boot 3.5, Spring Cloud 2025.0	Microservicios, gateway y BFF
Build	Maven (multimódulo)	Compilación dentro de Docker (multi-stage)
Frontend	React + Vite + TypeScript, MSAL	SPA con login de Microsoft
Contenedores	Docker 29, Docker Compose	Orquestación de 16 contenedores del backend (incluye RabbitMQ)
Base de datos	PostgreSQL 15 (uno por servicio), Flyway	Persistencia y migraciones
Nube	AWS EC2, Elastic IP, Security Groups (AWS Academy)	Infraestructura
Sistema operativo	Ubuntu Server 24.04 LTS	Ambas instancias
Proxy y TLS	Caddy 2 + Let's Encrypt, sslip.io	HTTPS automático sin dominio propio
Servidor web	nginx (alpine)	Servir el build del frontend
Identidad	Microsoft Entra ID (OAuth 2.0 / OIDC)	Autenticación y validación de tokens JWT
Mensajería	RabbitMQ 3.13 (contenedor con panel de administración)	Eventos asíncronos entre ms-message y ms-notification
Correo	Brevo SMTP	Envío de correos desde ms-notification
Observabilidad	GlitchTip (compatible Sentry), Spring Actuator	Errores y estado de salud
Control de versiones	Git + GitHub	Código fuente y despliegue con `git clone` / `git pull`

5. Despliegue del backend

El backend corre en una EC2 t3.large (2 vCPU, 8 GB RAM) con 30 GiB gp3, Ubuntu 24.04 y la IP elástica 32.201.134.233.

5.1 Preparación del repositorio. Se ajustó `docker-compose.yml` para producción (commit `8701bbd`):

- Se quitó el servicio `frontend`, que ahora vive en otra instancia.
- Se agregó `restart: unless-stopped` a todos los servicios, para que vuelvan a levantarse tras un reinicio del lab.
- `GLITCHTIP_DOMAIN` pasó a leerse desde `.env`.

5.2 Creación de la instancia. AMI Ubuntu Server 24.04 LTS, par de claves `vockey`, IP pública habilitada y un grupo de seguridad propio. Después se asignó y asoció una IP elástica.

5.3 Instalación de dependencias y swap.

```
sudo apt update && sudo apt install -y git
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker ubuntu
sudo fallocate -l 4G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

El swap de 4 GB evita que la compilación Maven de 7 módulos agote los 8 GB de RAM.

5.4 Código y variables de entorno. Se clonó el repositorio y se creó `.env` a partir de `.env.example`. Las claves `JWT_SECRET`, `INTERNAL_API_KEY` y `GLITCHTIP_SECRET_KEY` se generaron en el propio servidor con `openssl rand -hex 32`, por lo que nunca pasaron por Git.

```
git clone https://github.com/Ereneus/ClassFlowBack.git && cd ClassFlowBack
cp .env.example .env
sed -i "s|^JWT_SECRET=.*|JWT_SECRET=$(openssl rand -hex 32)|" .env
```

5.5 Construcción y arranque.

```
docker compose up -d --build
```

La construcción de las 7 imágenes tardó unos 327 s. Se crearon 7 volúmenes, la red `classflow-back-net` y las bases de datos quedaron en estado *Healthy*.

5.6 HTTPS con Caddy. Caddy corre como contenedor en la red del host. Obtiene el certificado de Let's Encrypt para `32-201-134-233.sslip.io` y redirige el tráfico al gateway en el puerto 8080.

```
printf "32-201-134-233.sslip.io {\n  reverse_proxy localhost:8080\n}\n" >
~/Caddyfile
docker run -d --name caddy --restart unless-stopped --network host \
  -v ~/Caddyfile:/etc/caddy/Caddyfile -v caddy_data:/data caddy:2
```

6. Despliegue del frontend

El frontend corre en una EC2 t3.medium (2 vCPU, 4 GB RAM) con 20 GiB gp3, Ubuntu 24.04 y la IP elástica 100.60.245.109. Su grupo de seguridad solo abre los puertos 22, 80 y 443.

Como Vite incrusta la configuración en el build, la URL de la API y los datos de Entra se pasan como argumentos de compilación de Docker. El `Dockerfile` del repositorio compila con Node 22 y sirve el resultado con nginx.

```
git clone https://github.com/Ereneus/ClassFlowFront.git && cd ClassFlowFront
docker build \
  --build-arg VITE_API_BASE_URL=https://32-201-134-233.sslip.io/api \
  --build-arg VITE_AUTH_MODE=entra \
  --build-arg VITE_MSAL_CLIENT_ID=<client ID de la app SPA> \
  --build-arg VITE_MSAL_TENANT_ID=<tenant ID> \
  --build-arg VITE_MSAL_API_SCOPE=api://<API client ID>/access_as_user \
  --build-arg VITE_MSAL_REDIRECT_URI=https://100-60-245-109.sslip.io \
  -t classflow-front .
docker run -d --name front --restart unless-stopped -p 127.0.0.1:8081:80 classflow-
front
```

El contenedor de nginx solo escucha en `127.0.0.1:8081`. Un segundo Caddy publica el sitio en `https://100-60-245-109.sslip.io` con su propio certificado de Let's Encrypt. Así, el único acceso desde internet es por HTTPS.

7. Autenticación con Microsoft Entra ID

Los usuarios inician sesión con su cuenta Microsoft y el backend solo acepta tokens emitidos por el tenant configurado. El gateway, el BFF y ms-auth corren con los perfiles Spring `docker,entra`.

1. El usuario pulsa *Iniciar sesión* en el frontend y MSAL abre el popup de `login.microsoftonline.com`.
2. Entra autentica al usuario y redirige a la URI registrada para la SPA.
3. El frontend obtiene un access token con el scope `access_as_user` de la API.
4. Cada llamada a la API envía el token en la cabecera `Authorization: Bearer`.
5. El gateway valida emisor (`ENTRA_ISSUER_URI`) y audiencia (`ENTRA_API_AUDIENCE`). Sin token responde **401**.
6. ms-auth busca la identidad por `tenant + oid`. Si no existe, la vincula con el usuario interno de mismo correo en el primer login.

Configuración en el portal de Azure. En el registro de la app SPA, plataforma *Aplicación de página única*, se agregaron las URIs `https://100-60-245-109.sslip.io` y `https://100-60-245-109.sslip.io/` (con y sin barra final), conservando `http://localhost:3000/` para desarrollo.

Por qué HTTPS era obligatorio. Entra solo acepta redirect URIs `https://` (excepto `localhost`). Además, una página HTTPS no puede llamar a una API HTTP por la regla de *mixed content*. Por eso ambas instancias necesitaron certificado.

8. Mensajería asíncrona con RabbitMQ

Al crear un mensaje, ms-message publica un evento y ms-notification lo consume para generar la notificación del destinatario. El broker es RabbitMQ 3.13 (imagen `rabbitmq:3.13-management-alpine`), que corre como un contenedor más del docker-compose en la EC2 del backend: el puerto AMQP 5672 solo es accesible dentro de la red Docker y el panel de administración queda en el puerto 15672.

Elemento	Nombre	Función
Exchange	`classflow.events` (topic)	Recibe los eventos de dominio
Routing key	`message.created`	Identifica el evento de mensaje nuevo
Cola	`classflow.messages.created`	Cola durable que consume ms-notification
Dead-letter exchange	`classflow.dlx`	Destino de mensajes fallidos
DLQ	`classflow.messages.dlq`	Guarda los eventos que fallaron 3 veces

Confirmación manual (ACK). ms-notification usa `acknowledge-mode=manual` y el listener decide de forma explícita el destino de cada mensaje:

Caso	Acción	Resultado
Procesado correctamente	`basicAck`	El mensaje se elimina de la cola
Evento inválido (sin destinatario o sin asunto)	`basicNack` sin reencolar	Va directo a la DLQ
Error al procesar, primera entrega	`basicNack` reencolando	Se reintenta una vez
Error al procesar, reentrega	`basicNack` sin reencolar	Va a la DLQ
JSON que no se puede convertir	Rechazo del contenedor	Va a la DLQ

Cumplimiento de la rúbrica. Los nombres de exchanges, colas y routing keys están centralizados en `application.properties` (`classflow.rabbitmq.*`) y se leen con `RabbitMqProperties`. La administración de colas, exchanges y bindings está encapsulada en `RabbitAdminService`; `RabbitMqAdminController` expone `POST/DELETE /api/rabbitmq/queues/{name}`, `/exchanges/{name}` y `/bindings/{exchange}/{queue}`, valida los parámetros con Bean Validation (`@NotBlank`, `@Size`, `@Pattern`) y responde 400, 409 o 503 según el caso.

Configuración. El usuario y la clave del broker se definen en el `.env` del servidor (`RABBITMQ_USER`, `RABBITMQ_PASSWORD`). ms-message y ms-notification se conectan a `amqp://…@rabbitmq:5672` y esperan a que el contenedor de RabbitMQ esté sano antes de arrancar. La lógica quedó cubierta por tests: 55 en ms-message y 36 en ms-notification, todos correctos.

Resultado de la prueba (5 de noviembre de 2026):

```
03:01:51.110  ms-message       Published RabbitMQ message-created event for message
26
03:01:51.747  ms-notification  Created notification from RabbitMQ message event 26
(ack)
03:02:07.567  ms-message       Published RabbitMQ message-created event for message
27
03:02:07.598  ms-notification  Created notification from RabbitMQ message event 27
(ack)
03:16:15.235  ms-message       Published RabbitMQ message-created event for message
```

```
28
03:16:15.250  ms-notification  Created notification from RabbitMQ message event 28
(ack)
```

Los tres mensajes se confirmaron con ACK: el primero en 637 ms (primera entrega tras el arranque) y los siguientes en 31 y 15 ms. Ninguno se reencoló ni terminó en la DLQ.

Prueba de un mensaje fallido (8 de noviembre de 2026). Para provocar un error real desde el frontend se detuvo la base de notificaciones (`docker compose stop notification-db`) y se envió un mensaje desde la vista de Mensajería. ms-message guardó el mensaje y publicó el evento; ms-notification no pudo crear la notificación, hizo `basicNack` reencolando en la primera entrega y `basicNack` sin reencolar en el reintento. El panel de RabbitMQ mostró el resultado:

Cola	Ready	Unacked	Interpretación
`classflow.messages.dlq`	1	0	El mensaje fallido quedó guardado en la Dead Letter Queue
`classflow.messages.created`	0	1	Un segundo mensaje entregado y aún sin confirmar mientras la base seguía apagada

El estado *Unacked* confirma que RabbitMQ no elimina un mensaje hasta que el consumidor lo confirma o lo rechaza de forma explícita. Al terminar la prueba se volvió a encender la base (`docker compose start notification-db`).

9. Problemas encontrados y soluciones

Problema	Causa	Solución
`ec2:RunInstances` denegado (*explicit deny* de la política `voclabs`)	AWS Academy solo permite tipos hasta `*.large`; se había elegido t3.xlarge	Se lanzó la instancia con t3.large y se agregó swap de 4 GB
Let's Encrypt: *Timeout during connect (likely firewall problem)*	El grupo de seguridad del backend solo tenía la regla SSH (22)	Se agregaron reglas HTTP (80) y HTTPS (443) al grupo asignado a la instancia
Error al guardar reglas: *Invalid rule description*	La descripción de la regla tenía tildes o caracteres no permitidos	Se dejó la descripción vacía
El frontend no puede usar Entra por HTTP	MSAL exige `https://` y el navegador bloquea *mixed content*	HTTPS en ambas EC2 con Caddy + Let's Encrypt y nombres sslip.io
La IP pública cambia al reiniciar el lab	AWS Academy detiene las instancias al cerrar la sesión	IP elástica en cada instancia: URLs, certificados y redirect URI estables
`docker compose up` intentaba construir `../frontend`	El compose original incluía el frontend	Se quitó ese servicio del compose del backend

Problema	Causa	Solución
ms-message publicaba pero ms-notification no registraba el evento 16	Un ms-notification local (Docker Desktop) usaba la misma cola de CloudAMQP y se llevó el mensaje	Se detuvo el entorno local; la cola quedó con 1 consumidor y el evento 17 llegó a la EC2
Comandos ejecutados en la instancia equivocada	Dos terminales abiertas, una por EC2	Verificar el prompt (`ip-172-31-43-251` backend, `ip-172-31-30-66` frontend)

10. Pruebas y evidencias

Las 8 validaciones realizadas el 5 de noviembre de 2026 resultaron correctas.

#	Prueba	Comando o acción	Resultado
1	Herramientas instaladas en el backend	`docker --version && git --version && free -h`	Docker 29.9.3, Git 2.53, 7.6 GiB RAM + 4 GiB swap
2	Construcción del backend	`docker compose up -d --build`	7 imágenes construidas, 82/82 recursos arriba, bases *Healthy*
3	Certificado del backend	`docker logs caddy`	`certificate obtained successfully` para `32-201-134-233.sslip.io`
4	API protegida	Abrir `https://32-201-134-233.sslip.io/`	Candado válido y HTTP 401: el gateway rechaza peticiones sin token
5	Certificado del frontend	`docker logs caddy` en la EC2 del frontend	`certificate obtained successfully` para `100-60-245-109.sslip.io`
6	Login con Microsoft Entra	Iniciar sesión en el frontend	Acceso al *Portal Docente*: 10 cursos, 20 estudiantes, 87 % de asistencia
7	Conexión a RabbitMQ	`docker compose logs ms-notification`	`Created new connection` a `rabbitmq:5672`
8	Flujo de eventos	Enviar un mensaje desde el frontend	Mensajes 26, 27 y 28 publicados y confirmados con ACK (15 a 637 ms)

Capturas sugeridas para anexar: consola EC2 con las dos instancias, grupos de seguridad, IPs elásticas, `docker compose ps`, dashboard con candado y la vista *Overview* del panel de RabbitMQ.

11. Seguridad, operación y mantenimiento

Medidas de seguridad aplicadas:

- Solo HTTPS hacia internet: el gateway (8080) y nginx (8081) no se exponen directamente; Caddy es la única entrada.
- Grupos de seguridad mínimos: solo 22, 80 y 443 abiertos; GlitchTip (8000) se abrirá únicamente para la IP del administrador.
- Bases de datos sin puertos publicados: solo accesibles dentro de la red Docker `classflow-back-net`.
- Secretos fuera del repositorio: `.env` está en `.gitignore` y las claves se generaron en el servidor.
- Autenticación delegada en Microsoft Entra ID, con validación de emisor y audiencia en cada petición.
- Contenedores de los microservicios ejecutados con un usuario sin privilegios (`USER app` en los Dockerfile).

Operación diaria en AWS Academy. Al iniciar el lab, las instancias arrancan y `restart: unless-stopped` levanta todos los contenedores, Caddy incluido. Los servicios Java tardan 2 a 3 minutos en quedar listos.

Actualizar el backend:

```
cd ~/ClassFlowBack && git pull && docker compose up -d --build
```

Actualizar el frontend: `git pull`, repetir el `docker build` con los mismos argumentos, `docker rm -f front` y volver a ejecutar el `docker run`.

Diagnóstico: `docker compose ps` para el estado y `docker compose logs --tail=100 <servicio>` para los logs.

12. Conclusiones y trabajo futuro

Se cumplieron los 6 objetivos: ClassFlow funciona en AWS con 16 contenedores de backend, frontend separado, HTTPS válido, login con Microsoft Entra ID y eventos asíncronos por RabbitMQ.

Los aprendizajes principales fueron tres. Primero, las restricciones del entorno (tipos de instancia de AWS Academy, IPs que cambian) condicionan el diseño tanto como el código. Segundo, la seguridad de identidad moderna obliga a tener HTTPS de punta a punta. Tercero, en una arquitectura con colas, cada entorno necesita su propio broker o *vhost*, porque los consumidores compiten por los mensajes.

Trabajo futuro:

- Configurar GlitchTip y cargar el DSN en `SENTRY_DSN`.
- Usar un broker RabbitMQ separado para el entorno local de desarrollo.
- Poner AWS API Gateway con JWT Authorizer como frontera pública, según la guía del proyecto.
- Mover las credenciales de PostgreSQL del compose al `.env` o a AWS Secrets Manager.
- Automatizar el despliegue con GitHub Actions (build, push de imágenes y `docker compose pull`).
- Usar un dominio propio en lugar de sslip.io.
- Migrar al clúster Kubernetes con los manifiestos ya incluidos en `k8s/`.